

10 · Extension 系统如何修改 Pi 行为

扩展发现、jiti 加载、注册表、事件钩子、Runtime 绑定与生命周期

分析版本：027a5847901b5dde30270abaa1041046cd2b4b55

核心结论

Extension 的能力来自“注册阶段 + 运行时绑定 + 事件总线”三部分。loader 用 jiti 加载 TS factory，并给它 ExtensionAPI；factory 调 pi.on/registerTool/registerCommand/registerShortcut/registerFlag 等把定义写入 Extension 对象。随后 ExtensionRunner/bindCore 把 action methods 绑定到当前 AgentSession。事件从输入、context、provider request、tool call 一直到 session lifecycle 都有拦截点。

1. 扩展文件从哪里来

自动发现位置包括 ~/.pi/agent/extensions 与可信项目的 .pi/extensions；CLI -e 可临时加载路径，package manager 还可从 npm/git 提供 extension source。ResourceLoader 负责把 package/settings/CLI 路径解析成最终 extension set。项目本地扩展受 project trust 约束。

2. 为什么 TypeScript 扩展不需要预编译

extensions/loader.ts 使用 jiti 加载 TS/JS。为了 Bun standalone binary 与 workspace/发布包都能工作，loader 还配置 virtualModules/aliases，把 pi-agent-core、pi-ai、pi-tui、pi-coding-agent 和 typebox 暴露给 extension。

3. Factory 加载阶段只有“注册”，没有完整 session action

createExtensionRuntime 起初给 sendMessage、appendEntry、setSessionName 等 action method 安装 throwing stub；这能阻止扩展在 factory 加载阶段错误使用尚未初始化的 Session。注册 Tool/Provider 等是允许的，真正 action 会在 Runner.bindCore 后替换。

4. ExtensionAPI 如何把注册写入 Extension 对象

API	写入/作用	运行影响
pi.on(event,handler)	extension.handlers Map	事件触发时按 handler 调用
registerTool	extension.tools Map	刷新 Agent active tool registry
registerCommand	extension.commands Map	/command 路由与 autocomplete
registerShortcut	extension.shortcuts Map	CustomEditor 输入优先级
registerFlag	extension.flags Map + runtime flagValues	CLI unknown long flags 二次解析
registerMessageRenderer	messageRenderers	自定义消息 TUI 展示
registerEntryRenderer	entryRenderers	custom SessionEntry 展示

registerProvider	pending/provider runtime	动态模型 Provider

5. 为什么扩展可以修改“几乎每一层”

因为事件钩子刻意布置在架构边界，而不是只在 UI 上：input 位于 string 输入边界；before_agent_start 位于 Agent run 前；context 位于 AgentMessage->Message 前；before_provider_headers/request 位于 Provider 请求边界；tool_call/tool_result 位于工具执行边界；session_before_compact/tree/switch 位于状态变更边界。

```
input
-> before_agent_start
-> agent_start / turn_start
-> context
-> before_provider_headers/request
-> provider response
-> tool_call -> tool_result
-> turn_end -> agent_end -> agent_settled
```

6. Event interception 与普通 observer 的区别

某些事件只是通知；某些事件允许返回值改变控制流。例如 input 可以 handled/transform；tool_call 可以 block；tool_result 可改结果；session_before_compact 可 cancel 或提供自定义 summary；before_agent_start 可改 system prompt。阅读扩展时必须区分“监听”与“拦截”。

7. 工具注册如何穿透到 Agent Core

Extension tool 最初是 Coding Agent ToolDefinition，AgentSession 合并内置/custom/extension definitions，选择 active set，再 wrapRegisteredTools/wrapToolDefinition 转为 AgentTool 写入 agent.state.tools。模型的 toolCall 最终仍由 agent-loop 的统一 prepare/validate/execute 流程执行，因此扩展工具不会绕过 core 的生命周期。

8. 命令为什么能绕过模型

AgentSession.prompt 在普通 input transform/skill expansion 之前先尝试 _tryExecuteExtensionCommand。命中命令后 handler 直接执行并 return，不构造 user AgentMessage。InteractiveMode 的 autocomplete 则从 extensionRunner.getRegisteredCommands 读取命令。因此扩展命令既是 UI 功能，也是业务路由。

9. Runtime stale 防护

session replacement 或 reload 后，旧的 extension ctx 可能引用已销毁 Session。createExtensionRuntime 有 assertActive/invalidate 机制，旧 ctx 被标记 stale 后继续使用会抛出明确错误。文档要求 newSession/fork/switchSession 后将后续工作放到 withSession 新 ctx，reload 后不要继续使用旧 ctx。

10. 生命周期与资源管理

扩展 factory 可能在仅执行 list-models 等场景被加载，却没有真正 session；因此文档明确不应在 factory 启动长期 timer/socket/process。长期资源应在 session_start 或实际 command/tool 时创建，并在 session_shutdown 幂等释放。

11. Reload 为什么需要清 cache

ResourceLoader.reload 在已加载过时调用 clearExtensionCache，然后重新解析设置/package/resource paths。extension cache 还按 cwd/generation 管理，避免跨目录错误复用模块。/reload 不只是重新渲染 UI，而是重建扩展和资源运行时。

12. 安全边界

- Extension 具有当前用户完整系统权限，本质上是可信插件代码，不是沙箱。
- 项目本地扩展必须经过 trust；非交互模式依赖 saved/default trust 或 --approve。
- 危险操作防护可以用 tool_call hook，但恶意扩展本身可直接访问 Node API，因此 trust 的对象是扩展代码本身。
- before_provider_request/headers 能看到敏感请求信息，安装第三方扩展需视为高权限行为。

13. 最适合扩展的点与不适合的点

需求	推荐扩展点	原因
阻止危险 bash/edit	tool_call / beforeToolCall	参数已结构化、执行尚未发生
给每轮补上下文	context 或 before_agent_start messages	不修改核心 Session 实现
自定义系统提示	before_agent_start	可按 turn 动态决定
自定义摘要	session_before_compact	保留 SessionManager checkpoint 机制
新模型服务	registerProvider/registerNativeProvider	复用 ModelRuntime/Provider abstraction
长期状态	appendEntry + session_start 重建	与 JSONL Session 同步

动态验证方案

实验 1：加载与注册

假设：factory 在 session_start 前完成注册

操作：写 async factory 延迟初始化并注册命令，session_start handler 检查其存在。

预期：startup 会等待 factory，命令在 session_start 时已经注册。

若不一致：若不等待，检查是否通过正确 loader 加载。

实验 2：stale ctx

假设：reload/session replacement 后旧 ctx 不可继续使用

操作：捕获旧 ctx，执行 reload 后尝试调用 action。

预期：assertActive 报 stale ctx 错误。
若不一致：若仍可使用，存在跨 session 状态泄漏风险。

实验 3：工具拦截

假设：extension tool 仍经过 Agent Core 统一校验和事件
操作：注册 TypeBox 工具，订阅 tool_call/tool_result/tool_execution_*。
预期：模型调用时出现完整 core + extension event 链。
若不一致：若工具直接绕过，检查是否是 command 中手动执行而非 LLM tool。

源码证据索引

文件	关键符号/用途	本报告结论
packages/coding-agent/src/core/extensions/loader.ts	jiti / virtualModules / createExtensionRuntime / createExtensionAPI	加载、注册与 runtime 初态
packages/coding-agent/src/core/extensions/types.ts	ExtensionAPI/Context/events/ ToolDefinition	扩展能力契约
packages/coding-agent/src/core/extensions/runner.ts	bindCore / event emit / command/tool access	运行期绑定与事件分派
packages/coding-agent/src/core/resource-loader.ts	extension path resolve / trust / reload cache	发现与重载生命周期
packages/coding-agent/src/core/agent-session.ts	input/before_agent_start/tool/session hooks	扩展如何进入核心控制流
packages/coding-agent/docs/extensions.md	生命周期图与使用约束	仓库文档交叉验证

证据状态：除非单独标注，本报告中的行为结论均来自上述固定 commit 的静态源码与仓库测试/文档交叉验证；未实际启动 Pi、未抓取真实网络请求，因此“运行结果已验证”项为空。

推荐阅读顺序

优先级	文件	阅读目的	精读
1	packages/coding-agent/docs/extensions.md	先建立事件全景	是
2	packages/coding-agent/src/core/extensions/loader.ts	看 factory/API/runtime 怎样生成	是
3	packages/coding-agent/src/core/extensions/runner.ts	看 event 与 core 如何绑定	是
4	packages/coding-agent/src/core/agent-session.ts	按 input/tool/compact 搜事件调用点	是
5	packages/coding-agent/examples/extensions/	最后用小扩展验证心智模型	否